1. Algoritmo para Determinar el Número de Componentes Conexas de un Grafo *(Presentación N° 3)*

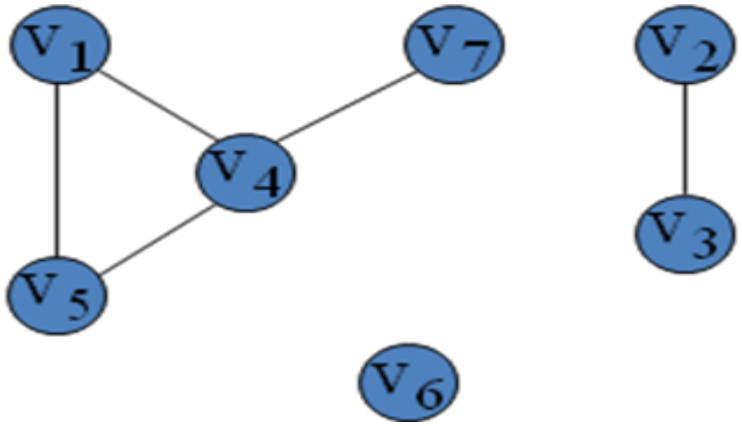
```
Algoritmo ComponentesConexas
Inicio
  Leer el número de vértices del grafo
  Lee los arcos (matriz de adyacencia)
  Se inicializa el conjunto (familia de conjunto) de las componentes conexas, asumiendo que cada vértice es una
  componente conexa
  Inicializa el número de componentes conexas, y se asumen que existen N componentes conexas al inicio.
  para cada uno de los arcos
        Se extrae un arco distinto del conjunto de los arcos: <v, w>
        Buscar la Componente Conexa a la que pertenece el vértice v y se almacena en la variable i
        Buscar la Componente Conexa a la que pertenece el vértice w y se almacena en la variable j
        si (i ≠ j) entonces    // vértices v y w del arco están en componentes distintas
            Se aplica la unión del conjunto donde fue ubicado el vértice v con el conjunto donde fue ubicado el vértice w
            El conjunto donde se ubico el vértice w se descarta (se coloca en ∅) // elimina la componente conexa j.
            Se decrementa la cantidad de componentes conexas existentes.
        fsi
  fin de cada uno de los arcos
Fin  // fin ComponentesConexas
```

Otro pseudocódigo para el mismo algoritmo

```
Algoritmo ComponentesConexas
constante
  N ← 100
tipo
  matrizAdy = arreglo [1..N][1..N] de entero
  vértice = entero
  arco = registro v, w: arco fregistro
variable
  A: matrizAdy
  C: arreglo [1..N] de conjuntos
  arc: arco
  n, p, i, j: entero
Inicio
  lecturasIniciales(n, A)   // Lee el número de vértices y los arcos del grafo
  Inicializa(n, C)   // C es inicializado de modo tal que cada vértice es una componente conexa
  p ← n   // Inicializa el número de componentes conexas en n
  mientras (existan_arcos) hacer
        arc ← leer_arco(n, A)
        i ← Buscar_ComponenteConexa_del_Vértice(arc.v, C)
        j ← Buscar_ComponenteConexa_del_Vértice(arc.w, C)
        si (i ≠ j) entonces    // vértices v y w del arco están en componentes distintas
          C[i] ← C[i] ∪ C[j]   //une las componentes conexas
          C[j] ← ∅   // elimina la componente conexa  j
          p ← p - 1
        fsi
  fmientras
Fin  // fin ComponentesConexas
```

Corrida en Frío

Grafo Ejemplo:



n ←
A ←

	Arc				C						
P	V	w	i	j	1	2	3	4	5	6	7

2. Algoritmo de las Ondas: Del Camino más Corto entre 'X' y 'Y' *(Presentación N° 3)*

Algoritmo CaminoMásCortoX_Y
// Información necesaria para el algoritmo:
// N es la cantidad de vértices
// Matriz de adyacencia del grafo
Inicio
leer el número de vértices del grafo: n y lee los arcos
leer los vértices: origen X y destino Y
Se marcan los vértices como no visitados, con un valor especial como por ejemplo -1
Búsqueda identifica si existe camino entre los vértices X y Y (Búsqueda_ hacia_adelante)
si (ExisteCam) **entonces** // Tiene sentido construir el camino
Se construye el camino hacia atrás entre estos dos vértices
sino
escribir ("No existe camino entre ", X, " y ", Y)
fsi
Fin // fin CaminoMasCortoX_Y //

Procedimiento Búsqueda_hacia_adelante()
Inicio
Se inicializa colocando el vértice origen X en la onda 0, el epicentro.
Marca al vértice X como visitado con el valor 0
Inicializa la onda 1, colocando los adyacentes de X
Inicializa el índice de ondas en 1, que es la onda actual.
si (verifica si existen vértices en la onda 1) **entonces**
marca los vértices de la onda 1 como visitados con el valor 1
mientras (no haya conseguido a Y, y todavía queden adyacentes) **hacer**
Se inicializa la siguiente onda
para cada vértice w ∈ a la Onda actual **hacer**
Se agrega a la próxima onda el conjunto de vértices adyacentes no marcados de w
si (La próxima Onda contiene elementos) **entonces**
marcar vértices, todos los vértices registrados en la próxima onda como visitados
fsi
fpara
i ← i + 1
fmientras
fsi
si (el vertice Y se encuentra en la onda actual) **entonces**
ExisteCam se marca como *verdadero*
sino
ExisteCam se marca como *falso*
fsi
FinProcedimiento // fin procedimiento Búsqueda_hacia_adelante //

Procedimiento Construcción_hacia_atrás()
Inicio
Colocar el vértice destino Y, en la posición de la última onda generada
para k ← i [decrementando] **hasta** 1 // realiza el proceso de devolverse, construcción hacia atrás
Se coloca en el camino los vértices perteneciente a la onda anterior.
fpara
escribir("El camino más corto, en cantidad de arcos, entre ", X , " y ", Y ," es: ")
para k ← 0 [incrementando] **hasta** i // Muestra los vértices del camino, en el orden correcto
Se muestra los vértices que deben recorrer para ir de X a Y
fpara
FinProcedimiento // fin procedimiento Construcción_hacia_atrás //

3. Algoritmo para Determinar el Conjunto de Circuitos Independientes de un Grafo *(Presentación N° 3)*

Algoritmo **Conjunto_Circuitos_Independientes**
Inicio
Leer el número de vértices del grafo: n
Leer los arcos y sus etiquetas
Se inicializa el conjunto de etiquetas de los arcos tratados, en vacio, ninguna etiqueta tratada.
Inicializa el número de rango circuito en 0: r = 0
mientras (AuxA, que son los arcos tratados, no hayan llegado a la totalidad de arcos, A) **hacer**
Tomar la etiqueta de un arco no tratado
Para la etiqueta del arco tomado, determinar los vértices de ese arco
si (verifica la existencia de camino entre los vértices del arco) **entonces**
escribir "circuito independiente encontrado"
Se incrementa el número de rango circuito: r = r + 1
fsi
Añade la etiqueta a al conjunto de etiquetas de arcos tratados
fmientras
escribir("El Rango Circuito de G es: ", r)
Fin // fin Conjunto_Circuitos_Independientes //

4. Algoritmo de Kruskal *(Presentación N° 4)*

//Para hallar el árbol expandido mínimo a partir de un grafo conexo

procedimiento **Alg_Kruskal**(Ady: matrizAdy; A: conj_etiquetas_de_arco; n: entero; var A_{min}: conj_etiquetas_de_arco)
// **Ady: matrizAdy** Contiene la matriz de adyacencia del grafo G
// **A, A_{min}: conjunto_etiquetas_de_arco** Almacenan etiquetas a_i de los arcos del grafo
// **n: entero** es el número de vértices de la corrida ($n \leq N$)
Inicio
variable
i: entero; // i es contador del número de arcos añadidos al subgrafo actual T^i
CT: real; // CT es el costo del subgrafo expandido T^i
AuxA: conjunto_etiquetas_de_arco; // AuxA es variable auxiliar, contiene inicialmente las etiquetas de A
ei: etiqueta_de_arco; // Almacenará la etiqueta a_i del arco que se esté procesando
$A_{min} \leftarrow \emptyset$; // inicializa el conj. de etiquetas de los arcos de T^i en vacío
// (el subgrafo expandido T^i comienza con sólo los vértices)
CT ← 0; // el costo de T^i se inicializa en 0
AuxA ← A; // se colocan las etiquetas de los arcos del grafo G en una variable auxiliar
i ← 0; // el contador de número de arcos añadidos a T^i se inicializa en 0
mientras (i < n – 1) **hacer** // mientras no se haya formado el árbol expandido mínimo
ei ← extraer_etiqueta_arco_min_costo(AuxA – A_{min});
si ¬(forma_circuito_al_añadir(ei, A_{min}, Ady)) **entonces** // verifica que no se forme circuito al añadir ei
$A_{min} \leftarrow A_{min} \cup \{ei\}$;
CT ← CT + C(ei);
i ← i + 1;
sino // si se forma un circuito al añadir ei a los arcos de A_{min} (arcos del subgrafo T^i)
AuxA ← AuxA – {ei};
fsi
fmientras
escribir("El conjunto de arcos de T_{min} es: ", A_{min} , " con un costo de ", CT)
FinProcedimiento // fin procedimiento Alg_Kruskal //

5. Algoritmo de Warshall *(Presentación N° 8)*

```
//Para determinar la alcanzabilidad en un grafo dirigido

procedimiento Alg_Warshall (AdyB: matrizAdyBool; n: entero; var E*: matrizAdyBool)
//AdyB: matrizAdyBool (Contiene la matriz de adyacencia booleana del grafo G)
//n: entero (Es el número de vértices de la corrida, n ≤ N)
//E*: matrizAdyBool (Contendrá la matriz de alcanzabilidad del grafo G)
Inicio
        variable
                i, j, r: entero; // 'i', 'j' y 'r' son variables índices para recorrer matrices

        para i ← 1 [incrementando] hasta n // para llenar la matriz E* con la matriz de Ady y la matriz Identidad
                para j ← 1 [incrementando] hasta n
                        E*[i, j] ← AdyB[i, j];
                        si (i = j) entonces
                           E*[i, j] ← 1;
                        fsi
                fpara
        fpara
        para r ← 1 [incrementando] hasta n // para señalar un vértice pivote
                para i ← 1 [incrementando] hasta n // para recorrer las filas de E*
                        para j ← 1 [incrementando] hasta n // para recorrer las columnas de E*
                                si (E*[i, j] = 0) entonces
                                        E*[i, j] ← E*[i, r] ∧ E*[r, j];
                                fsi
                        fpara
                fpara
        fpara
FinProcedimiento // fin procedimiento Alg_Warshall //
```

6. Algoritmo de Dijkstra *(Presentación N° 9)*

//Para hallar el costo de los caminos mínimos desde un vértice fuente a los demás vértices del grafo. Aplica a grafos dirigidos y pesados.

```
procedimiento Alg_Dijkstra (G: grafo; n: entero; var Costo: vector[1..n])
Inicio
    variable
        T: conj_vertices; // En T se van agregando los vértices ya tratados
        i: entero; // i es el contador de vértices

    T ←{1};
    Costo[1] ← 0;
    para i ← 2 [incrementando] hasta n
        si ( (1, i) ∈ A) entonces
            Costo[i] ← C(1, i);
        sino
            Costo[i] ← ∞;
        fsi
    fpara
    para i ← 1 [incrementando] hasta n - 1
        k ← vértice_menor_costo(V–T); // k es un vértice tal que Costo[k] es el menor
        T ← T ∪{k};
        si (T ≠ V) entonces
            para cada vértice w en (V –T) hacer
                si ( (k, w) ∈ A) entonces
                    si ( Costo[k] + C(k, w) < Costo[w] ) entonces
                        Costo[w] ← Costo[k] + C(k, w);
                    fsi
                fsi
            fpara
        fsi
    fpara
FinProcedimiento // fin procedimiento Alg_Dijkstra //
```

Algoritmo de Dijkstra Modificado *(Presentación N° 9)*

//Para hallar el costo de los caminos mínimos desde un vértice fuente a los demás vértices del grafo, además guarda los caminos. Aplica a grafos dirigidos y pesados.

```
procedimiento Alg_Dijkstra_Modificado(G: grafo; n: entero; var Costo, Cam: vector[1..n])
Inicio
    variable
        T: conj_vertices; // En T se van agregando los vértices ya tratados
        i: entero; // i es el contador de vértices

    inicializar Cam[i] ← 0; ∀i ∈ {1, 2, …, n} // Aquí se inicializan todos los caminos
    T ←{1};
    Costo[1] ← 0;
    Cam[1] ← 1;
    para i ← 2 [incrementando] hasta n
        si ( (1, i) ∈ A) entonces
            Costo[i] ← C(1, i);
            Cam[i] ← 1;
        sino
            Costo[i] ← ∞;
        fsi
    fpara
    para i ← 1 [incrementando] hasta n - 1
        k ← vértice_menor_costo(V–T); // k es un vértice tal que Costo[k] es el menor
        T ← T ∪{k};
        si (T ≠ V) entonces
            para cada vértice w en (V–T) hacer
                si ( (k, w) ∈ A) entonces
                    si ( Costo[k] + C(k, w) < Costo[w] ) entonces
                        Costo[w] ← Costo[k] + C(k, w);
                        Cam[w] ← k;
                    fsi
                fsi
            fpara
        fsi
    fpara
FinProcedimiento // fin procedimiento Alg_Dijkstra_Modificado //
```